

CNN From Scratch versus Transfer Learning: A Comparative Study on CIFAR-10

Aanya Singh

Deep Learning Course Assignment

Roll No: 23DCS002

Abstract—This report compares a small convolutional neural network trained from scratch against a ResNet-18 model fine-tuned via transfer learning, both on the CIFAR-10 image classification task. Both models are trained and evaluated on an identical data split for a fair comparison. We report test accuracy, macro and per-class precision/recall/F1-score, macro AUC-ROC, parameter counts, training time, and convergence speed, and visualize the learned first-layer kernels of both networks. The fine-tuned ResNet-18 substantially outperforms the custom CNN, reaching 94.19% test accuracy against 76.17% for the network trained from scratch, converging in far fewer epochs. We relate this gap to network depth, the vanishing-gradient problem, receptive field size, and the role of residual (skip) connections.

Index Terms—convolutional neural networks, transfer learning, CIFAR-10, ResNet-18, image classification

I. INTRODUCTION

Convolutional Neural Networks (CNNs) are the standard architecture for image classification, but their performance depends heavily on depth, receptive field, and the availability of large amounts of labeled training data. This assignment implements two contrasting approaches to the same 10-class CIFAR-10 classification problem: (1) a small CNN with four convolutional layers, trained entirely from scratch, and (2) a ResNet-18 backbone pretrained on ImageNet and fine-tuned end-to-end on CIFAR-10. The goal is to quantify how much transfer learning and architectural depth improve performance, training efficiency, and the quality of learned low-level features, and to relate the observed differences to concepts covered in class: depth, the vanishing-gradient problem, receptive field, and residual (skip) connections.

II. EXPERIMENTAL SETUP

A. Dataset

Both models were trained and evaluated on the CIFAR-10 dataset (10 classes, 32×32 RGB images) [1]. To keep training time tractable, 40% of the 50,000 training images were used ($n = 20,000$), of which 10% were held out for validation. The exact same image indices were used for both models to guarantee a fair, apples-to-apples comparison. The full 10,000-image official test set was used for final evaluation of both models.

B. Custom CNN Architecture

The custom network consists of four convolutional layers (32, 64, 128, 128 filters, all 3×3 with padding 1), each

TABLE I
DATA SPLIT USED FOR BOTH MODELS

Split	Number of images
Training	18,000
Validation	2,000
Test	10,000

followed by batch normalization, a ReLU activation, and max-pooling after layers 1, 2 and 4. This is followed by a dropout layer ($p = 0.3$) and two fully connected layers (256 units, then 10 output classes). The network was trained from randomly initialized weights using the Adam optimizer (learning rate 10^{-3}) with a cosine-annealing learning-rate schedule, for 15 epochs, using standard CIFAR-10 data augmentation (random crop with padding 4, random horizontal flip).

C. Pretrained / Transfer Learning Architecture

For the deeper architecture we fine-tuned ResNet-18 [2], initialized with ImageNet-pretrained weights. The final fully connected layer was replaced with a 10-class linear layer, and all layers were fine-tuned end-to-end (not frozen). Input images were resized to 224×224 and normalized with ImageNet statistics to match the pretrained weights' expected input distribution. The model was trained with the Adam optimizer (learning rate 10^{-4}) and a cosine-annealing schedule for 8 epochs.

D. Experiment Tracking

Both training runs were logged to a public Weights & Biases (WandB) project, tracking per-epoch training/validation loss and accuracy, together with final test-set metrics and confusion matrix images. The runs are named `custom_cnn_cifar10` and `resnet18_finetune_cifar10`. Project link: <https://wandb.ai/xanax/cnn-vs-transfer-cifar10>.

III. RESULTS

A. Training and Validation Curves

Fig. 1 and Fig. 2 show the training/validation loss and accuracy curves for the custom CNN and the fine-tuned ResNet-18, respectively. The custom CNN improves steadily but plateaus around 78% training accuracy after 15 epochs, with validation accuracy tracking closely behind, indicating limited overfitting but also a lower performance ceiling. The pretrained ResNet-18, in contrast, reaches over 90% validation

accuracy within the very first epoch and continues to improve, with training accuracy approaching 99.8% by epoch 8, though the growing train/validation gap in later epochs indicates the onset of mild overfitting on the reduced 18k-image training subset.

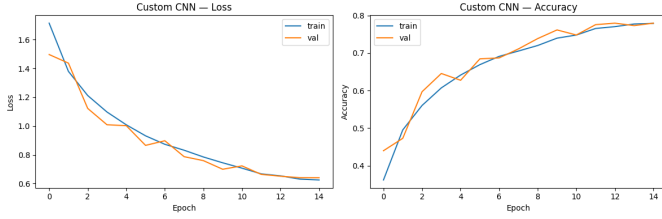


Fig. 1. Custom CNN — training/validation loss (left) and accuracy (right) over 15 epochs.

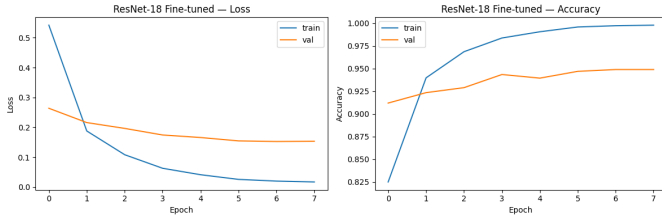


Fig. 2. Fine-tuned ResNet-18 — training/validation loss (left) and accuracy (right) over 8 epochs.

B. Confusion Matrices

Fig. 3 and Fig. 4 show the test-set confusion matrices. The custom CNN shows substantial confusion between visually similar animal classes, most notably *cat* vs. *dog* (146 cats misclassified as dogs) and *bird/deer/horse* cross-confusions, as well as confusion between *plane*, *ship* and *truck* classes that share background/color statistics. The fine-tuned ResNet-18’s confusion matrix is markedly more diagonal-dominant; the largest remaining error mode is still *cat/dog* confusion (82 cats predicted as dogs), but at roughly half the rate of the custom CNN, and cross-vehicle confusions are nearly eliminated.

C. Evaluation Metrics

Table II summarizes overall test-set performance, model size, training time, and convergence speed (number of epochs to first reach 60% validation accuracy). Table III reports per-class precision, recall and F1-score for both models.

The macro AUC-ROC (one-vs-rest) confirms the pattern seen in the hard-prediction metrics: the custom CNN separates classes by confidence score reasonably well (0.9715) but is clearly surpassed by the fine-tuned ResNet-18 (0.9975), which is close to a perfect ranking of positive versus negative examples across all ten classes. Convergence speed shows the most dramatic gap: the pretrained model reaches 60% validation accuracy after only 1 epoch of fine-tuning, versus 4 epochs for the model trained from scratch, directly demonstrating the practical benefit of starting from ImageNet-pretrained weights.

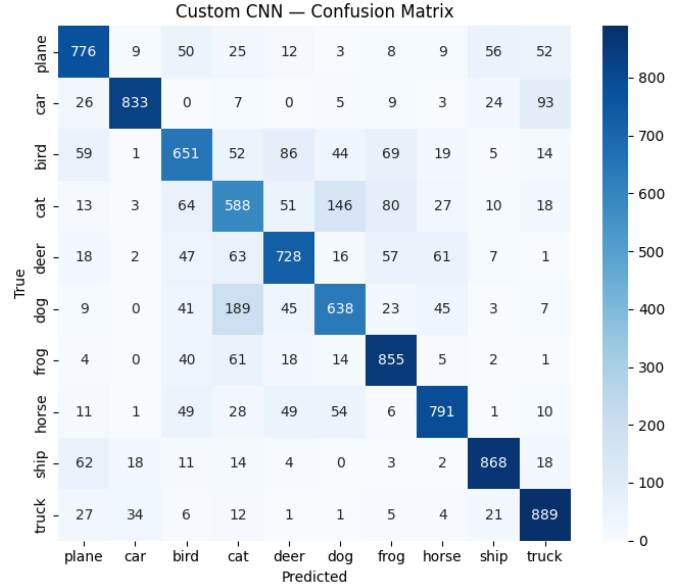


Fig. 3. Custom CNN — test-set confusion matrix.

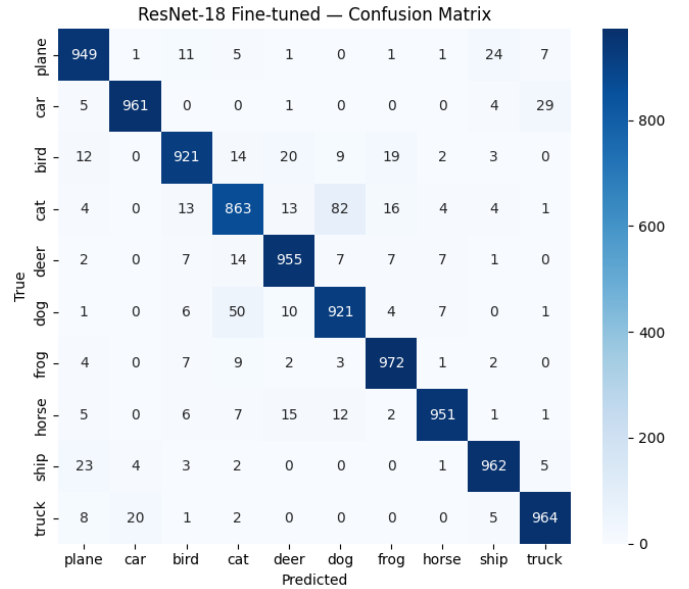


Fig. 4. Fine-tuned ResNet-18 — test-set confusion matrix.

TABLE II
OVERALL COMPARISON OF THE TWO MODELS

Metric	Custom CNN	ResNet-18 (FT)
Test Accuracy	0.7617	0.9419
Macro Precision	0.7629	0.9420
Macro Recall	0.7617	0.9419
Macro F1-score	0.7614	0.9418
Macro AUC-ROC (OvR)	0.9715	0.9975
Trainable Parameters	768,650	11,181,642
Training Time (s)	133.4	577.2
Epochs to reach val. acc. ≥ 0.60	4	1

TABLE III
PER-CLASS PRECISION / RECALL / F1-SCORE

Class	Custom CNN			ResNet-18 (FT)		
	P	R	F1	P	R	F1
plane	0.77	0.78	0.77	0.94	0.95	0.94
car	0.92	0.83	0.88	0.97	0.96	0.97
bird	0.68	0.65	0.66	0.94	0.92	0.93
cat	0.57	0.59	0.58	0.89	0.86	0.88
deer	0.73	0.73	0.73	0.94	0.95	0.95
dog	0.69	0.64	0.66	0.89	0.92	0.91
frog	0.77	0.85	0.81	0.95	0.97	0.96
horse	0.82	0.79	0.80	0.98	0.95	0.96
ship	0.87	0.87	0.87	0.96	0.96	0.96
truck	0.81	0.89	0.85	0.96	0.96	0.96
Macro avg	0.76	0.76	0.76	0.94	0.94	0.94

D. Kernel Visualization

Fig. 5 and Fig. 6 show the first-convolutional-layer kernels learned by each model. The custom CNN’s kernels are dominated by fairly uniform, saturated color-block patterns with little fine spatial structure — a signature of a randomly initialized filter bank trained on a comparatively small amount of data (18k images) with no low-level inductive bias beyond what the architecture itself provides. In contrast, the ResNet-18 kernels, learned by pretraining on the much larger and more diverse ImageNet corpus, show clearer oriented-edge and elongated stripe-like patterns, together with a few color-opponent (green/magenta) blob detectors — the classic Gabor-like low-level filters widely reported for CNNs trained on large image corpora. This qualitative difference in first-layer representations is consistent with the large gap in downstream classification performance: the pretrained network starts from filters that already generalize well to natural images, whereas the custom CNN must learn everything, including basic edge detection, from a much smaller CIFAR-10 subset.

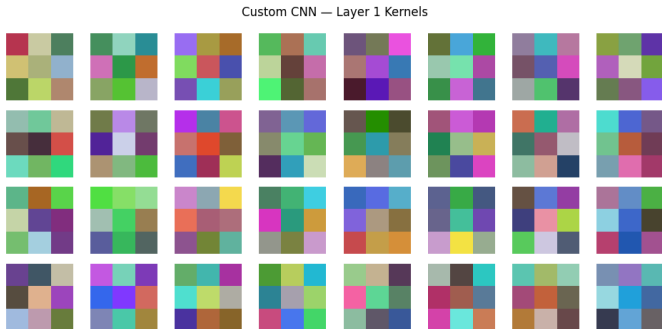


Fig. 5. Custom CNN — first-layer convolutional kernels.

IV. COMPARISON AND DISCUSSION

The fine-tuned ResNet-18 achieved 94.19% test accuracy versus 76.17% for the custom four-layer CNN, despite both models being trained on the exact same 18,000-image data split. This gap is consistent with what depth and architectural design contribute to CNNs. ResNet-18’s residual (skip) connections let gradients flow directly across blocks during

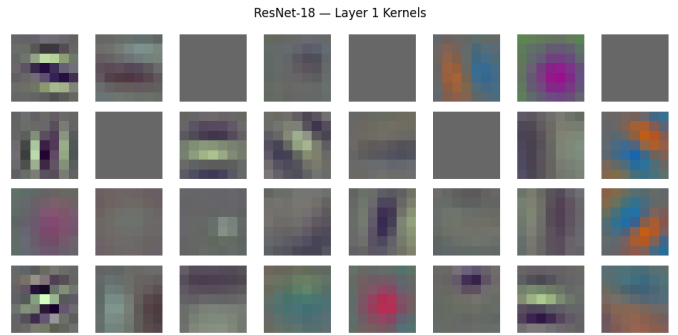


Fig. 6. Fine-tuned ResNet-18 — first-layer convolutional kernels.

backpropagation, mitigating the vanishing-gradient problem that limits how deep a plain CNN like ours can be trained effectively; our custom model avoids this issue only by staying shallow, at four convolutional layers. Because ResNet-18 is considerably deeper, units in its later layers have a much larger effective receptive field, letting the network capture more global, part-based structure (e.g., an animal’s overall body shape) rather than only local textures and edges — visible qualitatively in the cleaner, more diverse first-layer kernels reported in Section III-D. ResNet-18 also starts from ImageNet-pretrained weights, so fine-tuning only has to adapt already general-purpose visual features to CIFAR-10’s ten classes; this is why it reaches the 0.60 validation-accuracy threshold in a single epoch versus four for the model trained from scratch, and why its training loss falls almost to zero within eight epochs. The trade-off is parameter count and per-epoch compute: ResNet-18 has roughly 11.18 million trainable parameters versus 0.77 million for the custom CNN, and its 224×224 input resolution makes each fine-tuning epoch considerably more expensive (577.2 s total training time vs. 133.4 s for the custom CNN over roughly half as many epochs). The remaining common error mode for both models — confusing cats and dogs — reflects genuine visual similarity between the two classes at low resolution rather than a depth-related weakness, since it persists, albeit at a reduced rate, even in the much stronger ResNet-18.

V. CONCLUSION

This study confirms that, for a moderately sized subset of CIFAR-10, transfer learning from a deeper, ImageNet-pretrained architecture substantially outperforms a small CNN trained from scratch, across every metric considered: accuracy, macro F1, macro AUC-ROC, and convergence speed. The observed differences are consistent with the theoretical roles of network depth, residual connections, and receptive field size discussed in class, and are further corroborated by qualitative differences in the learned first-layer kernels of the two models. The main cost of the transfer-learning approach is a roughly $14 \times$ increase in trainable parameters and a higher per-epoch computational cost due to the larger input resolution required by the pretrained backbone.

ACKNOWLEDGMENT

Both training runs, validation curves, and evaluation metrics discussed in this report were logged to a public Weights & Biases project, available at <https://wandb.ai/xanax/cnn-vs-transfer-cifar10>.

REFERENCES

- [1] A. Krizhevsky, V. Nair, and G. Hinton, "CIFAR-10 (Canadian Institute for Advanced Research)," University of Toronto, [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems (NIPS)*, 2012.
- [4] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *arXiv preprint arXiv:1409.1556*, 2015.